

DevPilot: Autonomous RAG Code Navigator

End-to-End Architectural Reference, Code Ingestion, Chunking & pgvector Integration

BACKEND Spring Boot 4.1.1	FRONTEND Next.js 16 + React 19	VECTOR STORE PostgreSQL 16 (pgvector)	AI MODEL Google Gemini 3.6 Flash
------------------------------	-----------------------------------	--	-------------------------------------

1. System Architecture & Core Pipeline

DevPilot is an enterprise-grade Retrieval-Augmented Generation (RAG) platform that indexes GitHub codebases into vector embeddings and powers interactive conversational navigation. The entire lifecycle operates in 5 deterministic stages:

1. Authentication	2. Repo Sync	3. Ingestion & AST	4. pgvector Store	5. RAG Chat (SSE)
GitHub OAuth2 flow with AES-256 encrypted access token persistence.	Recursive tree traversal fetching repo metadata via GitHub REST API.	Filtering vendor/binary files, language detection, and token chunking.	Batch vector embeddings stored in PostgreSQL with cosine HNSW index.	Query similarity retrieval, prompt injection, and SSE token streaming.

2. Data Division: AST Filtering, Chunking & Vectorization

Source code cannot be naively chunked like plain text. DevPilot enforces a multi-tier code ingestion pipeline:

Step	Component / Class	Algorithmic Details & Operational Mechanics
1. File Filtering	<code>CodeFileFilter.java</code>	Excludes vendor dirs (node_modules, dist, .git), skips binaries, enforces < 100 KB limit, and detects programming language extensions.
2. Context Tagging	<code>CodeChunker.java</code>	Prepends <code>// File: path/to/file.ext\n</code> to each file string before tokenization so chunks retain symbol context even when split.
3. Token Splitting	<code>TokenTextSplitter</code>	Splits code into ~200 token semantic segments (calculated as <code>chunkSize / 4</code>) and attaches metadata: <code>repold</code> , <code>filePath</code> , <code>language</code> , <code>chunkIndex</code> .
4. Batch Embedding	<code>IndexingService.java</code>	Batches chunks into groups of 16 (<code>VECTOR_BATCH_SIZE = 16</code>) to maximize throughput while preventing HTTP payload timeouts.
5. Rate Limit Guard	<code>safeAddVectors()</code>	Catches Gemini API HTTP 429 quota exhaustion errors and automatically parses retry delays, performing exponential backoff for up to 5 attempts.

3. PgVector Database Schema & Cosine Retrieval

Embeddings are stored in PostgreSQL using the pgvector extension. Vector similarity search uses the cosine distance operator (<=>) filtered strictly by repold to prevent cross-tenant data leakage.

```
-- PostgreSQL pgvector Storage Schema
CREATE TABLE vector_store (
  id UUID PRIMARY KEY,
  content TEXT NOT NULL,           -- Chunk source code + filepath header
  metadata JSONB NOT NULL,        -- repoId, filePath, language, chunkIndex
  embedding VECTOR(3072) NOT NULL -- High-dimensional Gemini embedding
);

-- Cosine Similarity Query in CodeContextRetriever.java
SELECT id, content, metadata, (embedding <=> :queryVector) AS distance
FROM vector_store
WHERE metadata->>'repoId' = :repoId
ORDER BY distance ASC LIMIT 8;
```

4. Real-Time RAG Chat & SSE Streaming Execution

When the user queries the chat interface (e.g. *'Explain how authentication works'*), the request flows through the real-time RAG stream:

1. Dispatch	2. Context Search	3. Prompt Fusion	4. SSE Token Stream
Client sends POST /api/chat/sessions/{id}/messages with JSON payload { content }.	CodeContextRetriever embeds question and fetches top-8 nearest chunks from pgvector.	ChatPromptBuilder wraps retrieved chunks into system prompt instructions with citation rules.	ChatStreamHandler calls Gemini 3.6 Flash and emits tokens via SseEmitter in real-time.

5. Master Class & Method Responsibility Matrix

Layer	Class	Key Methods	Technical Responsibility
Security	CryptoConfig.java	textEncryptor()	AES-256 token encryption for GitHub OAuth2 tokens stored in PostgreSQL.
Security	GithubOAuth2UserService	loadUser()	Handles OAuth callback, provisions User entity, and encrypts OAuth access tokens.
GitHub	GithubApiClient.java	getRepoTree(), getFile()	Fetches recursive repository file trees and raw file blobs from GitHub REST API.
Indexing	IndexingService.java	doIndex(), safeAddVectors()	Async pipeline coordinating tree walk, batching 16 chunks, and 429 quota backoff.
RAG AI	CodeContextRetriever.java	retrieveContext()	Runs cosine similarity search on pgvector and extracts file citation spans.
RAG AI	ChatStreamHandler.java	stream(), appendToken()	Calls Gemini 3.6 Flash via Spring AI, streaming tokens to client via SseEmitter.
REST API	ChatController.java	sendMessage(), getMessages()	Exposes SSE streaming endpoint producing text/event-stream for real-time UI.
Frontend	use-chat.ts	useStreamChat()	React hook reading SSE chunks, updating live stream text, and updating TanStack cache.